

SPOTIFY DATA SET ANALYSIS USING PYTHON

KAUSTUBH NAYAK

RAJ VERMA

SWARA VEDAK

Table of content

1. Abstract

The music industry is constantly evolving, and as the world's leading music streaming service, Spotify needs to stay ahead of the curve. Understanding what makes a song popular and how it changes over time is critical for creating successful music recommendations and driving user engagement. In this report, we explore how time series analysis can be used to identify trends, patterns, and changes in song features that contribute to a song's popularity on Spotify. We focus on the use of linear regression to analyze the relationship between independent variables, such as tempo, key, and release date, and a song's weekly streaming count. Our analysis shows that certain song features, such as upbeat tempo and major key, are strongly associated with higher streaming counts. We also explore how these relationships change over time, providing valuable insights into the evolving preferences of Spotify users. Overall, our report demonstrates the importance of time series analysis for understanding what makes a song popular and how it changes over time, and provides actionable insights for improving music recommendations and driving user engagement on Spotify.

2. Introduction:

The music industry is a highly competitive and rapidly changing landscape, where the popularity of songs is a crucial factor in an artist's success. However, understanding what makes a song popular and how those features change over time is a complex and challenging task. With the help of time series analysis, we can identify key features that contribute to a song's popularity and track how they change over time.

In this report, we will explore the use of time series analysis in identifying the factors that make a song popular and how those factors change over time. We will begin by defining time series analysis and explaining its relevance in the music industry. We will then examine the key features that contribute to a song's popularity, such as tempo, key, and lyrics, and how those features have evolved over time.

Next, we will discuss various methods used for analyzing time series data, including statistical modeling and machine learning algorithms. We will also explore how these methods can be used to identify trends, patterns, and changes in song features that contribute to a song's popularity.

Finally, we will review case studies where time series analysis has been used to identify the factors that contribute to a song's popularity and how those factors have changed over time. These case studies will illustrate the power and potential of time series analysis in the music industry, highlighting its ability to track trends and predict the success of new songs.

Overall, this report aims to provide a comprehensive overview of the use of time series analysis in identifying the factors that make a song popular and how those factors change over time. By exploring the various methods, techniques, and case studies, we hope to demonstrate the value of time series analysis in the music industry and its potential to shape the future of music.

3. Objectives

- To explore the various methods used for analyzing time series data, including statistical modeling and machine learning algorithms.
- To understand how these methods can be used to identify trends, patterns, and changes in song features that contribute to a song's popularity on Spotify.
- To focus on the use of linear regression for analyzing the relationship between independent variables, such as tempo, key, and release date, and a song's weekly streaming count.
- To provide valuable insights into the evolving preferences of Spotify users and how these relationships change over time.
- To demonstrate the importance of time series analysis for understanding what makes a song popular and how it changes over time, and provide actionable insights for improving music recommendations and driving user engagement on Spotify.

4. Methods used and Implementation

To implement time series analysis in Python for Spotify data, we can follow these steps:

1. Collect and clean the data: The first step is to collect the Spotify data and clean it. The data should include features such as tempo, key, duration, artist, and streaming count. Once we have the data, we need to clean it by removing missing values and duplicates.
2. Convert data into time series: Next, we need to convert the data into a time series format, where each data point is a weekly streaming count for a particular song.
3. Visualize the time series: We should visualize the time series to understand the trends, patterns, and changes in the streaming counts over time. This can be done using libraries such as Matplotlib or Seaborn.
4. Decompose the time series: We can use the Seasonal Decomposition of Time Series (STL) method to decompose the time series into its three components: trend, seasonal, and remainder. This can help us identify seasonal patterns in song features that contribute to a song's popularity.
5. Apply linear regression: We can use linear regression to model the relationship between independent variables, such as tempo, key, and release date, and a song's weekly streaming count. This can help us identify which song features are most strongly associated with a song's popularity.
6. Evaluate the model: Once we have a linear regression model, we need to evaluate its performance using metrics such as Mean Squared Error (MSE) or R-squared. This can help us determine how well the model predicts a song's popularity based on its features.
7. Use the model to predict popularity: Finally, we can use the linear regression model to predict the popularity of new songs based on their features. This can help us improve music recommendations and drive user engagement on Spotify.

Overall, implementing time series analysis in Python for Spotify data can provide valuable insights into what makes a song popular and how it changes over time, and can help us improve music recommendations and drive user engagement on the platform.

5. Methodologies :

There are several methodologies that can be used in Python for time series analysis of Spotify data to identify what makes a song popular and how it changes over time. Here are some of the commonly used methodologies:

1. ARIMA (Auto Regressive Integrated Moving Average) model: ARIMA is a popular time series modeling technique that can be used to model the relationship between a song's features and its streaming count. ARIMA models can be used to forecast future streaming counts based on historical data and can help us identify trends and patterns in the data.
2. Seasonal Decomposition of Time Series (STL) method: STL is a method that can be used to decompose a time series into its three components: trend, seasonal, and remainder. This can help us identify seasonal patterns in the data and understand how these patterns affect a song's popularity.
3. Linear regression: Linear regression is a simple but powerful technique that can be used to model the relationship between a song's features and its streaming count. Linear regression models can help us identify which song features are most strongly associated with a song's popularity and can help us predict the popularity of new songs based on their features.
4. Machine learning algorithms: There are several machine learning algorithms that can be used to analyze time series data, including Random Forest, XGBoost, and LSTM (Long Short-Term Memory). These algorithms can be used to identify complex patterns in the data and can help us make accurate predictions about a song's popularity based on its features.
5. Time series clustering: Time series clustering is a technique that can be used to group similar songs based on their streaming counts over time. This can help us identify subgroups of songs that are popular for different reasons and can help us make more targeted music recommendations.

Overall, these methodologies can be used in Python for time series analysis of Spotify data to identify what makes a song popular and how it changes over time. Depending on the specific research question and data available, one or more of these methodologies may be appropriate.

6. Model used:

Using linear regression to predict what makes a song popular and how its features change over time on Spotify data is a common and effective method. Linear regression can help us identify which song features are most strongly associated with a song's popularity and can help us predict the popularity of new songs based on their features.

To use linear regression for this analysis, we first need to identify which song features we want to include as predictors in our model. These might include things like tempo, energy, danceability, and acousticness, among others. Once we have identified our predictors, we can use linear regression to model the relationship between these features and a song's streaming count.

One common approach is to use a multiple linear regression model, where we include multiple predictors in our model. This can help us identify which features are most strongly associated with a song's popularity and can help us make more accurate predictions about a song's future popularity.

To implement linear regression in Python, we can use the scikit-learn library, which provides a wide range of tools for machine learning and data analysis. We can use the `LinearRegression` class in scikit-learn to build a linear regression model, and we can use the `fit` method to fit the model to our Spotify data. Once we have fitted our model, we can use it to make predictions about the popularity of new songs based on their features.

Overall, using linear regression to predict what makes a song popular and how its features change over time on Spotify data is a powerful and effective approach that can help us make more informed decisions about music production, promotion, and marketing.

7. REQUIREMENTS AND IMPORTS

Pandas is an open source Python package that is most widely used for data science/data analysis and machine learning tasks. It is built on top of another package named NumPy, which provides support for multi-dimensional arrays. As one of the most popular data wrangling packages NumPy can be used to perform a wide variety of mathematical operations on arrays. It adds powerful data structures to Python that guarantee efficient calculations with arrays and matrices and it supplies an enormous library of highlevel mathematical functions that operate on these arrays and matrices Matplotlib is a library in Python that enables users to generate visualizations like histograms, scatter plots, bar charts, pie charts and much more. Seaborn is a visualization library that is built on top of Matplotlib. It provides data visualizations that are typically more aesthetic and statistically

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from tqdm import tqdm
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_squared_log_error
```

8. About our code:

Problem Statement 1

A. What makes a song popular :

a. Preparatory Functions

```
import warnings
warnings.filterwarnings("ignore")

sns.set_style("darkgrid")

from IPython.display import Markdown, display, HTML
def printmd(string):
    display(Markdown(string))

def prin(DataFrame):
    with pd.option_context('display.max_rows', 8,
                           'display.max_columns', None,
                           'display.width', 1000,
                           'display.precision', 3,
                           'display.colheader_justify', 'center'):
        display(DataFrame)

def normalize(df):
    result = df.copy()
    exc = ["Year", "year", "name", "release_date"]
    for feature_name in df.columns:
        if feature_name not in exc:
            # Min-Max Scale If it Is Not the Year Column
            print(feature_name)
            max_value = df[feature_name].max()
            min_value = df[feature_name].min()
            result[feature_name] = (df[feature_name] - min_value) / (max_value - min_value)
    return result
```

b. Data import

```
▶ global df
df=pd.read_csv("tracks.csv")
fd=pd.read_csv("artists.csv")

prin(df)
fd.tail(10)
```

c. Data pre-processing

```
[ ] xdf=xdf.copy()
xdf=xdf.drop(columns=["id","id_artists","explicit","time_signature"])
```

```
[ ] xdf.sort_values(by="popularity",ascending=False)
```

	name	popularity	duration_ms	artists	release_date	danceability	energy	key	loudness	mode	speechiness	acousticness	instrumentalness	liveness	valence	tempo
93802	Peaches (feat. Daniel Caesar & Giveon)	100	198082	[Justin Bieber, Daniel Caesar, Giveon]	2021-03-19	0.677	0.6960	0	-6.181	1	0.1190	0.3210	0.000000	0.420	0.464	90.030
93803	drivers license	99	242014	[Olivia Rodrigo]	2021-01-08	0.585	0.4360	10	-8.761	1	0.0601	0.7210	0.000013	0.105	0.132	143.874
93804	Astronaut In The Ocean	98	132780	[Masked Wolf]	2021-01-06	0.778	0.6950	4	-6.865	0	0.0913	0.1750	0.000000	0.150	0.472	149.996
92810	Save Your Tears	97	215627	[The Weeknd]	2020-03-20	0.680	0.8260	0	-5.487	1	0.0309	0.0212	0.000012	0.543	0.644	118.051
92811	telepatía	97	160191	[Kali Uchis]	2020-12-04	0.653	0.5240	11	-9.016	0	0.0502	0.1120	0.000000	0.203	0.553	83.970
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
23410	O Pardesi Musafir	0	198053	[Suraiya, Lata Mangeshkar]	1949-01-01	0.682	0.3050	1	-8.558	1	0.0578	0.9240	0.000000	0.119	0.910	152.850
23409	Mere Dil Mein Aao Aur Bas Jao	0	171133	[Suraiya]	1949-01-01	0.527	0.2030	7	-14.199	1	0.0667	0.9900	0.000012	0.129	0.688	170.733
23408	Divisions, Op. 21: Variation 9, Toccata I	0	46720	[Benjamin Britten, Leon Fleisher, Seiji O...	1949	0.441	0.2420	6	-28.776	1	0.0323	0.9290	0.955000	0.116	0.528	132.060
23407	John Henry	0	189973	[Lead Belly]	1949	0.593	0.7480	2	-6.514	0	0.4060	0.8880	0.021800	0.680	0.732	123.634
444118	Kevin Barry	0	139493	[The Clancy Brothers, Tommy Makem, Paddy...	2015-06-29	0.431	0.0325	9	-17.961	1	0.0344	0.9670	0.000000	0.355	0.391	140.434

```
[ ] xdf.dtypes
xdf['Year'] = pd.to_datetime(xdf['release_date'], format='%Y-%m-%d')
xdf['year'] = pd.to_datetime(xdf['Year'],errors='coerce').dt.year
```

xdf

	name	popularity	duration_ms	artists	release_date	danceability	energy	key	loudness	mode	speechiness	acousticness	instrumentalness	liveness	valence	tempo	Year	year
0	Carve	6	126903	[UIU]	1922-02-22	0.645	0.4450	0	-13.338	1	0.4510	0.674	0.744000	0.1510	0.1270	104.851	1922-02-22	1922
1	Capítulo 2.16 - Banquero Anarquista	0	98200	[Fernando Pessoa]	1922-06-01	0.695	0.2630	0	-22.136	1	0.9570	0.797	0.000000	0.1480	0.6550	102.009	1922-06-01	1922
2	Vivo para Quererte - Remasterizado	0	181640	[Ignacio Corsini]	1922-03-21	0.434	0.1770	1	-21.180	1	0.0512	0.994	0.021800	0.2120	0.4570	130.418	1922-03-21	1922
3	El Prisionero - Remasterizado	0	176907	[Ignacio Corsini]	1922-03-21	0.321	0.0946	7	-27.961	1	0.0504	0.995	0.918000	0.1040	0.3970	169.980	1922-03-21	1922
4	Lady of the Evening	0	163080	[Dick Haymes]	1922	0.402	0.1580	3	-16.900	0	0.0390	0.989	0.130000	0.3110	0.1960	103.220	1922-01-01	1922
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
586667	云与海	50	258267	[阿YueYue]	2020-09-26	0.560	0.5180	0	-7.471	0	0.0292	0.785	0.000000	0.0648	0.2110	131.896	2020-09-26	2020
586668	blind	72	153293	[ROLE MODEL]	2020-10-21	0.765	0.6630	0	-5.223	1	0.0652	0.141	0.000297	0.0924	0.6860	150.091	2020-10-21	2020
586669	What They'll Say About Us	70	187601	[FINNEAS]	2020-09-02	0.535	0.3140	7	-12.823	0	0.0408	0.895	0.000150	0.0874	0.0663	145.095	2020-09-02	2020

- For the sake of processing power we will perform on last lakh records

```
[ ] xdf=xdf.sort_values('Year')
    xdf=xdf[-100000:]
```

```
%%script false --no-raise-error

plt.figure(figsize=(14,8))
plt.title("Four Most Popular Features' Trend Over Time", fontdict={"fontsize": 15})

lines = ["acousticness", "loudness", "energy", "valence", "instrumentalness", "liveness", "danceability"]
# for line in tqdm(lines, position=0, desc="Loading:", leave=False, colour='green', ncols=80):

for line in tqdm(lines, position=0, desc="Loading:", leave=False, colour='green', ncols=80):
    ax = sns.lineplot(x='Year', y=line, data=xdf, errorbar=("ci", False))

plt.ylabel("value")
plt.legend(lines)
```

Couldn't find program: 'false'

```
import plotly.express as px
%time

#for line in tqdm(lines, position=0, desc="Loading:", leave=False, colour='green', ncols=80):
fig = px.line(xdf, x="Year", y=lines, title="Features of Songs")
fig.show()
```

CPU times: total: 0 ns
Wall time: 0 ns

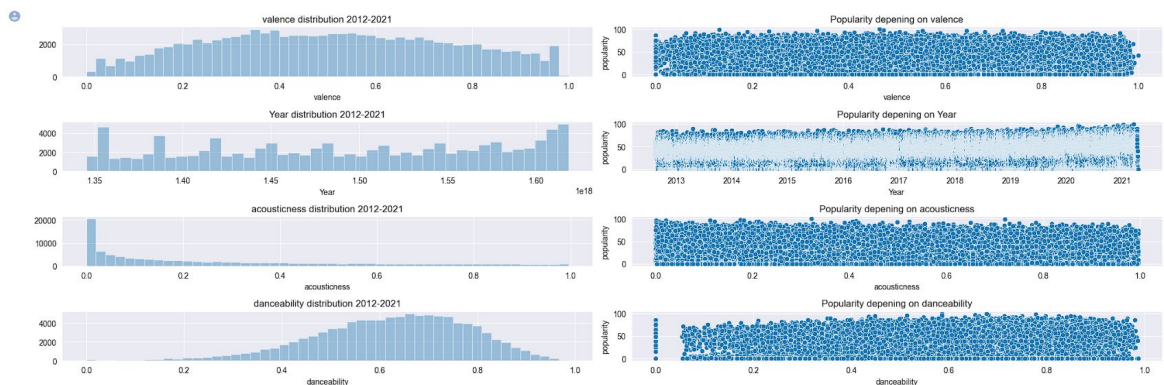
Do any of these directly affect popularity?

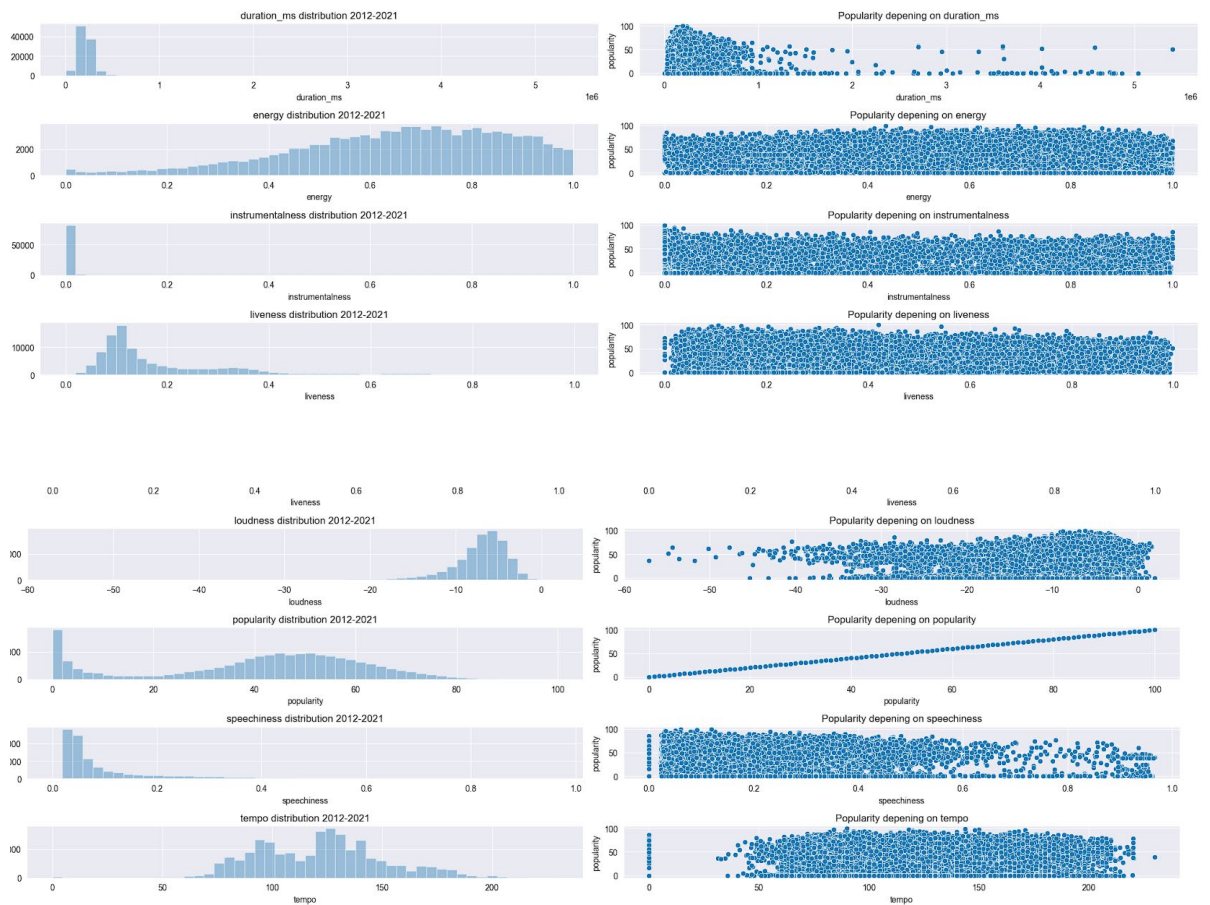
- Left with respect to **Years**
- Right with respect to **Popularity**

```
[ ] plt.figure(figsize=(20,20))
count=1
xy_columns = ['valence', 'Year', 'acousticness', 'danceability', 'duration_ms', 'energy', 'instrumentalness', 'liveness', 'loudness', 'popularity', 'speechiness', 'tempo']
x_columns = ['valence', 'Year', 'acousticness', 'danceability', 'duration_ms', 'energy', 'instrumentalness', 'liveness', 'loudness', 'speechiness', 'tempo']

for xvar in xy_columns:
    plt.subplot(12,2, count);
    count += 1
    plt.gca().set_title(xvar + ' distribution 2012-2021')
    sns.distplot(a=xdf[xvar], kde=False)
    plt.subplot(12,2,count)
    count +=1
    plt.gca().set_title('Popularity depening on ' + xvar)
    sns.scatterplot(x=xdf[xvar], y=xdf.popularity)
    #sns.kdeplot(data=data_num[[xvar, 'popularity']], hue="kind", fill=True)
    #mean_df = df.groupby([xvar])[ 'popularity'].mean().to_frame().reset_index()
    #sns.scatterplot(x=mean_df[xvar], y=mean_df.popularity, data=mean_df)

#plt.subplots_adjust(space = 0.5)
plt.tight_layout()
```





- While LHS gives us some insight
- The correlation with RHS is weak in most cases
- Outliers hold impact in some areas
- Data still isnt pre processed

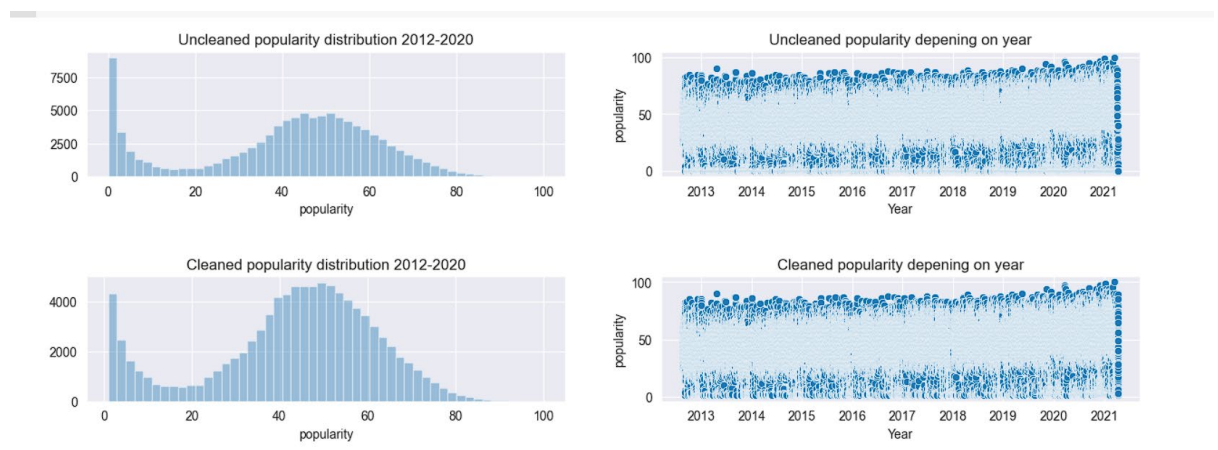
- Checking for Null Values

Since our target variable is popularity, we can remove all songs that have 0 popularity

```
# Let's remove 0 values rows for popularity
xdfp = xdf[xdf['popularity'] != 0]

# Popularity uncleaned
plt.figure(figsize=(15,5))
plt.subplot(2,2,1)
plt.gca().set_title('Uncleaned popularity distribution 2012-2020')
sns.distplot(a=xdf.popularity, kde=False)
plt.subplot(2,2,2)
plt.gca().set_title('Uncleaned popularity depening on year')
sns.scatterplot(x=xdf.Year, y=xdf.popularity)

# Popularity cleaned
plt.subplot(2,2,3)
plt.gca().set_title('Cleaned popularity distribution 2012-2020')
sns.distplot(a=xdfp.popularity, kde=False)
plt.subplot(2,2,4)
plt.gca().set_title('Cleaned popularity depening on year')
sns.scatterplot(x=xdfp.Year, y=xdfp.popularity)
plt.subplots_adjust(hspace = 0.8)
```



After removing 0 popularity songs, distribution has slightly improved, but not significantly, But its not as if the popularity was 0 because there was data missing hence we will proceed with the original dataset

```
# Proportion of 0 for x_columns
for col in x_columns:
    print('Zeroes in', col, ': ', round((len(xdf)- np.count_nonzero(xdf[col], axis=0))/len(xdf)*100,2), '%')
```

```
Zeroes in valence : 0.15 %
Zeroes in Year : 0.0 %
Zeroes in acousticness : 0.02 %
Zeroes in danceability : 0.15 %
Zeroes in duration_ms : 0.0 %
Zeroes in energy : 0.0 %
Zeroes in instrumentalness : 45.52 %
Zeroes in liveness : 0.01 %
Zeroes in loudness : 0.0 %
Zeroes in speechiness : 0.15 %
Zeroes in tempo : 0.15 %
```

```
#Really zero values
```

```
print("Is it 0 when tempo is 0?")
for col in x_columns:

    print( str(col)+' : ' + str(xdf[xdf['tempo']==0].equals(xdf[xdf[col]==0])))
```

```
Is it 0 when tempo is 0?
valence: False
Year: False
acousticness: False
danceability: True
duration_ms: False
energy: False
instrumentalness: False
liveness: False
loudness: False
speechiness: True
tempo: True
```

```
# Find some examples (most popular ones) of zero tempo songs
data_zero_tempo = xdf[xdf['tempo'] == 0].sort_values(by='popularity', ascending=False)
printmd('***# of songs with 0 bpm:*** ' + str(data_zero_tempo.shape[0]))
printmd('***Examples for 0 bpm songs:***')
display(data_zero_tempo[['name', 'artists', 'Year', 'tempo', 'speechiness', 'danceability', 'popularity']].head(10))
```



```
xdf=xdf[~xdf["name"].isin(data_zero_tempo.name)]
```


of songs with 0 bpm: 149
 Examples for 0 bpm songs:

	name	artists	Year	tempo	speechiness	danceability	popularity
89913	Clean White Noise - Loopable with no fade	['Erik Eriksson', 'White Noise Baby Sleep', 'W...	2017-01-08	0.0	0.0	0.0	86
90054	White Noise - 500 hz	['Granular']	2017-10-13	0.0	0.0	0.0	78
90104	White Noise - 145 hz	['Granular']	2017-10-13	0.0	0.0	0.0	77
91234	Soft Brown Noise	['High Altitude Samples']	2018-11-15	0.0	0.0	0.0	76
115632	Deep Sleep Recovery Noise	['Water Sound Natural White Noise']	2020-02-25	0.0	0.0	0.0	75
88262	Brown Noise - Loopable with No Fade	['White Noise Meditation', 'Lullaby Land', 'Wh...	2015-05-26	0.0	0.0	0.0	72
119442	White Noise Crashing Waves	['Bruce Brus']	2019-08-09	0.0	0.0	0.0	72
90578	Pure Brown Noise - Loopable with no fade	['Erik Eriksson', 'White Noise for Babies', 'W...	2017-01-08	0.0	0.0	0.0	72
177682	Clean & Pure White Noise	['Zenibly']	2018-08-01	0.0	0.0	0.0	70
89518	Box Fan Rest Sound	['Fan Sounds']	2016-11-11	0.0	0.0	0.0	69

These soundtracks are popular because it seems to have a positive effect and helps achieve deep sleep or even for relaxation.

- Albeit popular it doesn't align with our problem statement
- These will be removed

○ Relation with Mean of Popularity

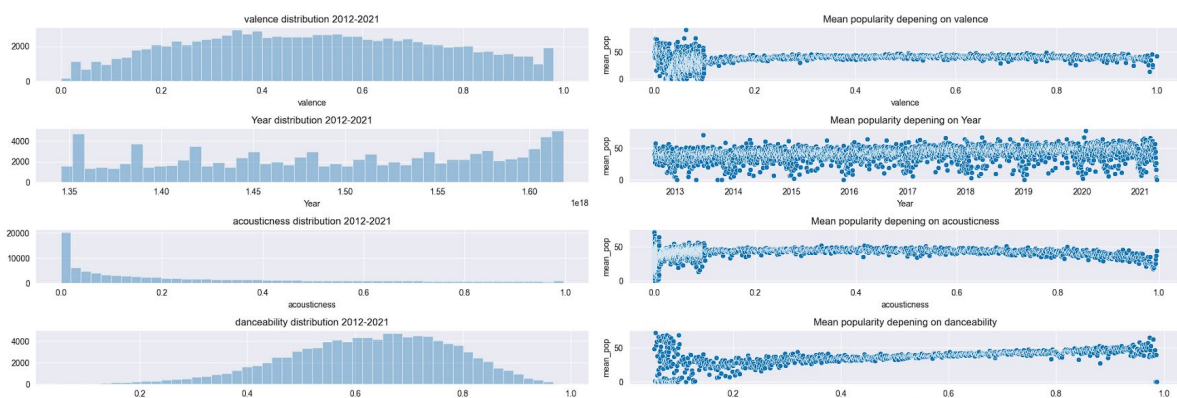
- LHS is same as before
- But we compare mean of popularity instead of popularity directly


```
plt.figure(figsize=(20,20))
count=1

for xvar in x_columns:
    plt.subplot(12,2, count)
    count += 1
    plt.gca().set_title(xvar + ' distribution 2012-2021')
    sns.distplot(a=xdf[xvar], kde=False)
    plt.subplot(12,2,count)
    count +=1
    plt.gca().set_title('Mean popularity depening on ' + xvar)

    mean_df = xdf.groupby([xvar])['popularity'].mean().to_frame().reset_index()
    mean_df['mean_pop'] = mean_df['popularity']
    sns.scatterplot(x=mean_df[xvar], y=mean_df.mean_pop)

plt.subplots_adjust()
plt.tight_layout()
```



We can see some linear and non linear trends for popularity

With respect to Mean popularity

When value is low	When its distributed	When value is high
valence	danceability	Year
liveness	energy	loudness
acousticness	Instrumentalness	
duration	tempo	
Speechiness		

```
[ ] year_avg = xdf[["acousticness","danceability","energy", "instrumentalness",
                    "liveness","tempo", "valence", "loudness", "speechiness", "year"]].groupby("year").mean().sort_values(by="year").reset_index()

tdf = xdf[["acousticness","danceability","energy", "instrumentalness",
            "liveness","tempo", "valence", "loudness", "speechiness", "year", "Year"]].sort_values(by="year").reset_index()

year_avg.head(15)
```

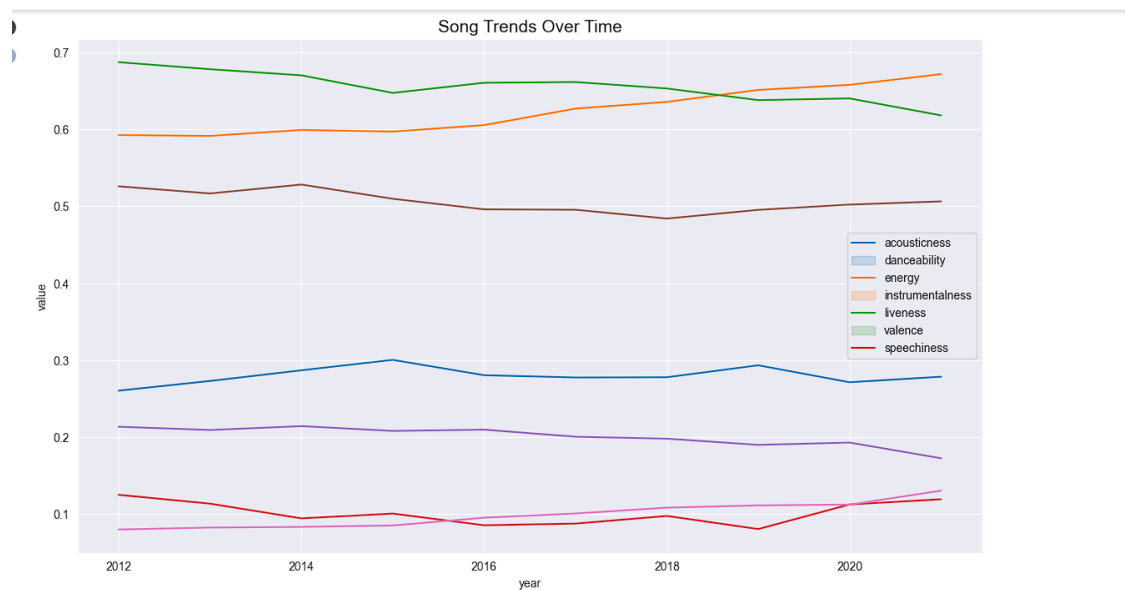
	year	acousticness	danceability	energy	instrumentalness	liveness	tempo	valence	loudness	speechiness
0	2012	0.260413	0.592552	0.687219	0.125085	0.213501	122.752127	0.525923	-7.078832	0.079999
1	2013	0.272997	0.591411	0.678079	0.113578	0.209301	123.233240	0.516646	-7.231258	0.082543
2	2014	0.286874	0.599207	0.670131	0.094457	0.214288	122.648701	0.528189	-7.098744	0.083419
3	2015	0.300406	0.597033	0.647296	0.100690	0.208095	121.511649	0.509726	-7.549708	0.085232
4	2016	0.280481	0.605445	0.660493	0.085509	0.209758	121.648680	0.496027	-7.229879	0.095298
5	2017	0.277536	0.626874	0.661419	0.087672	0.200527	121.704836	0.495469	-7.145313	0.100823
6	2018	0.277844	0.635628	0.653057	0.097601	0.198016	121.899125	0.484016	-7.367706	0.108375
7	2019	0.293353	0.651156	0.637927	0.080586	0.189949	122.291988	0.495333	-7.406817	0.111343
8	2020	0.271324	0.657745	0.640120	0.112545	0.192945	122.473887	0.502196	-7.518874	0.112348
9	2021	0.278531	0.671637	0.618095	0.119295	0.172582	120.777717	0.506308	-7.895880	0.130437

```
plt.figure(figsize=(14,8))
plt.title("Song Trends Over Time", fontdict={"fontsize": 15})

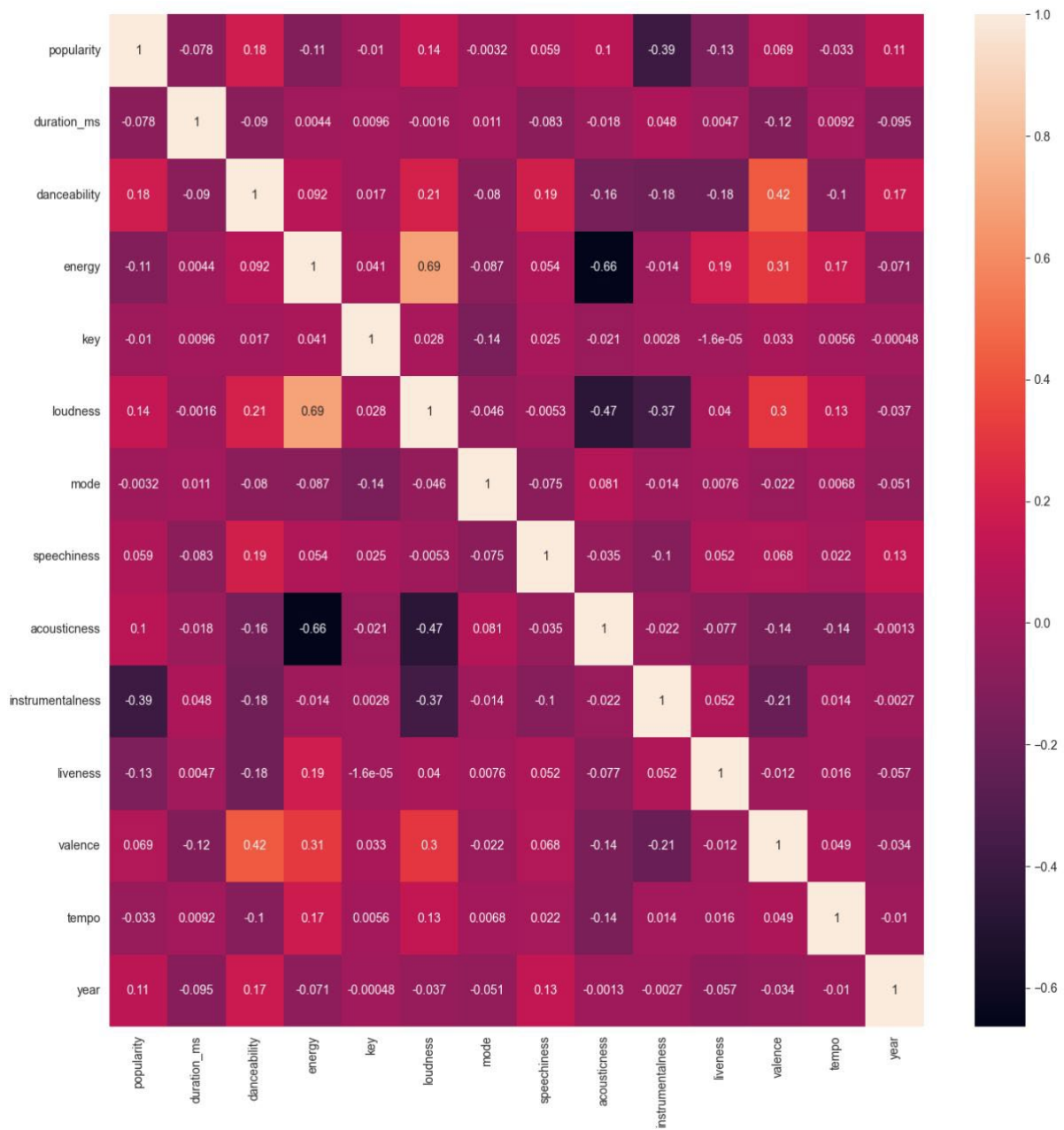
#without Loudness
lines = ["acousticness", "danceability", "energy",
         "instrumentalness", "liveness", "valence", "speechiness"]

for line in lines:
    ax = sns.lineplot(x='year', y=line, data=year_avg)

plt.ylabel("value")
plt.legend(lines)
```



d. Co-relation between each factor of the data



Insights

- acoustictness: correlated also negatively with loudness, energy and danceability and year
- instrumentalness: similar (but less signal) to acoustictness correlates also negatively with loudness, energy and danceability and year)

Positive Influence to popularity

- year
- energy
- loudness
- danceability

Year Might signify the effect of social Media on songs

e. Modelling, Training and Testing

▼ Prepare Data

```
[ ] #Lets Run a simple Linear Regression with popularity as target

train, test = train_test_split(xdf, test_size=0.3, shuffle=False)
training_features = ["acoustictness","danceability","energy", "instrumentalness",
                    "liveness","tempo", "valence", "loudness", "speechiness","year"]
target = 'popularity'
train_X = train[training_features]
train_Y = train[target]
test_X = test[training_features]
test_Y = test[target]
```

f. Applying model

Run Initial Model

```
[ ] #Model with 0 Tuning

model = LinearRegression()
model.fit(train_X, train_Y)
train_pred_Y = model.predict(train_X)
test_pred_Y = model.predict(test_X)
train_pred_Y = pd.Series(train_pred_Y.clip(0, train_pred_Y.max()), index=train_Y.index)
test_pred_Y = pd.Series(test_pred_Y.clip(0, test_pred_Y.max()), index=test_Y.index)

rmse_train = np.sqrt(mean_squared_error(train_pred_Y, train_Y))
msle_train = mean_squared_log_error(train_pred_Y, train_Y)
rmse_test = np.sqrt(mean_squared_error(test_pred_Y, test_Y))
msle_test = mean_squared_log_error(test_pred_Y, test_Y)

print('rmse_train:',rmse_train,'msle_train:',msle_train)
print('rmse_test:',rmse_test,'msle_test:',msle_test)
```

rmse_train: 17.8628306330308 msle_train: 1.205231349882275
rmse_test: 21.92740199765009 msle_test: 1.7473860767525375

```
xyz=pd.DataFrame({'coef':model.coef_, 'category':test_X.columns}).sort_values(by = 'coef', ascending = False).set_index('category')
display(xyz)
```

coef	
category	
danceability	11.367667
acousticness	2.798013
year	1.097700
loudness	0.836030
tempo	-0.011075
valence	-1.854156
speechiness	-1.996166
liveness	-7.728195
energy	-17.505816
instrumentalness	-25.469655

From the coefficients we can see that Most Popular are effected by

- instrumentalness
- Energy
- remarkably high danceability

Negative coefficients are not bad, they also indicate relationship, This may not align with common sense but it is simply what the Model thinks

```

▶ #Validate?
fig, ax = plt.subplots(figsize=(13, 7))
ax = sns.scatterplot(x=test_Y, y=test_pred_Y)
sns.lineplot(x=test_Y, y=test_Y, color='red', ax=ax)
ax.set_xlabel('Y_test')
ax.set_ylabel('Y_test_pred')
ax.set_title('y_test vs. y_test_pred', fontsize=14, color='red')
plt.show()

```



This model is not the most suited to make inferences off yet, but it works to some degree

○ Problem Statement 2

Feature Change over time

▼ Normalize

There is a clear need for normalizing these features, since its impossible to compare otherwise

```
▶ zdf = normalize(year_avg)
   tdf = normalize(tdf)
```

```
ⓘ acousticness
   danceability
   energy
   instrumentalness
   liveness
   tempo
   valence
   loudness
   speechiness
   index
   acousticness
   danceability
   energy
   instrumentalness
   liveness
   tempo
   valence
   loudness
   speechiness
```

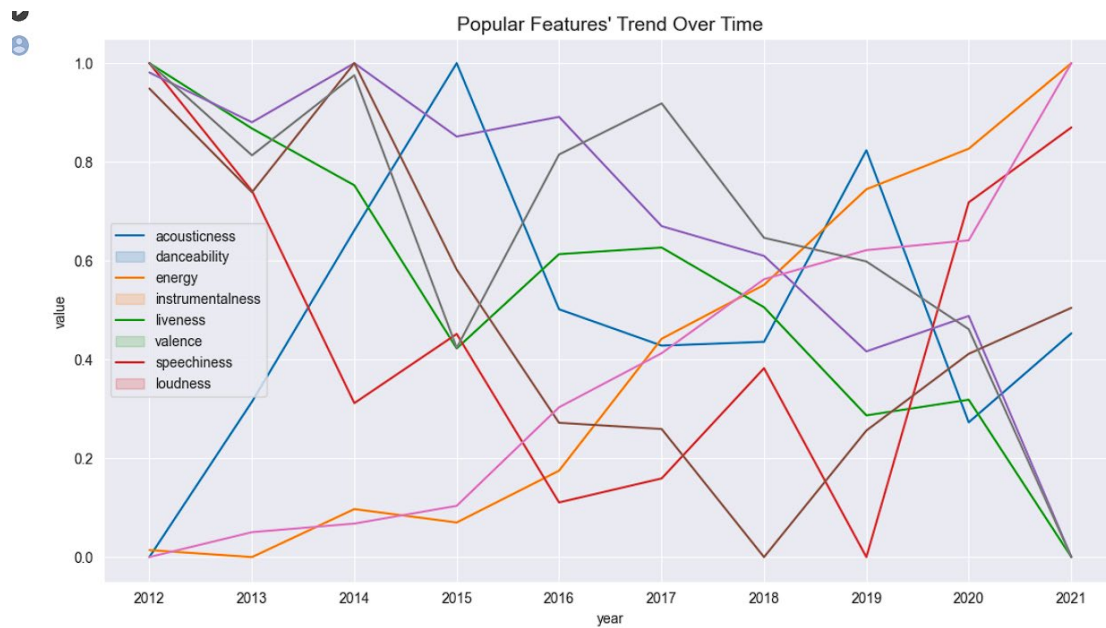
```
▶ # Create a simple line plot for initial analysis
plt.figure(figsize=(13,7))

plt.title("Popular Features' Trend Over Time", fontdict={"fontsize": 14})

lines = ["acousticness","danceability","energy",
         "instrumentalness", "liveness", "valence", "speechiness","loudness"]

for line in lines:
    ax = sns.lineplot(x='year', y=line, data=zdf)
    |

plt.ylabel("value")
plt.legend(lines)
```



Convert Data to Stationary Time Series

```

zdf.index
zdf.year=pd.to_datetime(zdf.year, format='%Y')
zdf.set_index('year', inplace=True)
zdf.isna().sum()

tdf.index
#tdf.year=pd.to_datetime(tdf.year, format='%Y')
tdf.set_index('Year', inplace=True)
tdf.isna().sum()

```

```

index          0
acousticness   0
danceability    0
energy         0
instrumentalness 0
liveness       0
tempo         0
valence        0
loudness       0
speechiness    0
year          0
dtype: int64

```



```

df_ts = []
for column in zdf:
    #Create separate dataframes for each category
    df = zdf[column]
    df = df.to_frame()
    df_ts.append(df)

df_ts[0].head()

```



acoustiness

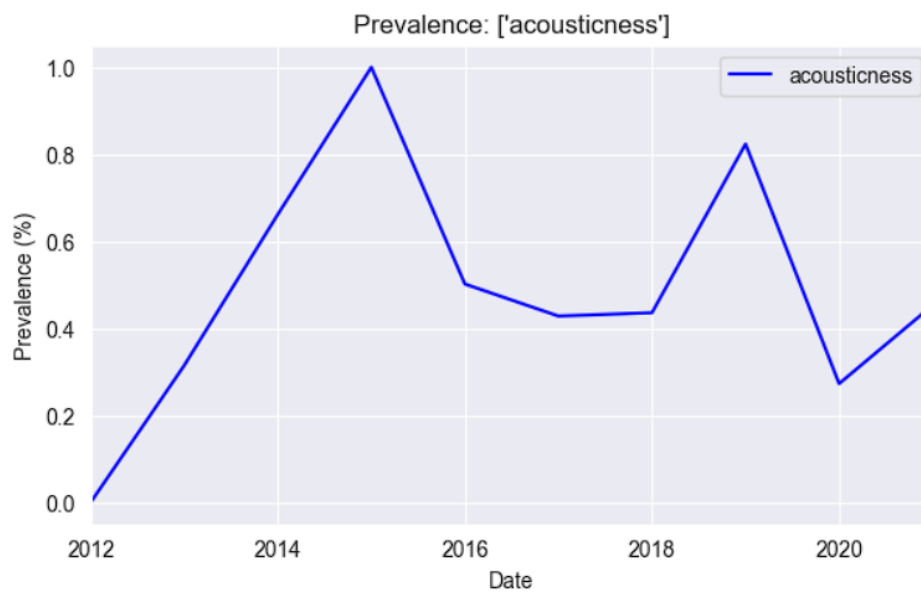
year

2012-01-01	0.000000
2013-01-01	0.314652
2014-01-01	0.661637
2015-01-01	1.000000
2016-01-01	0.501781

```

] for i in range(len(df_ts)):
    df_ts[i].plot(figsize=(7,4), color = 'b')
    plt.title(f'Prevalence: {df_ts[i].columns.to_list()}')
    plt.xlabel('Date')
    plt.ylabel('Prevalence (%)')
    plt.legend(loc='best')
    plt.show()

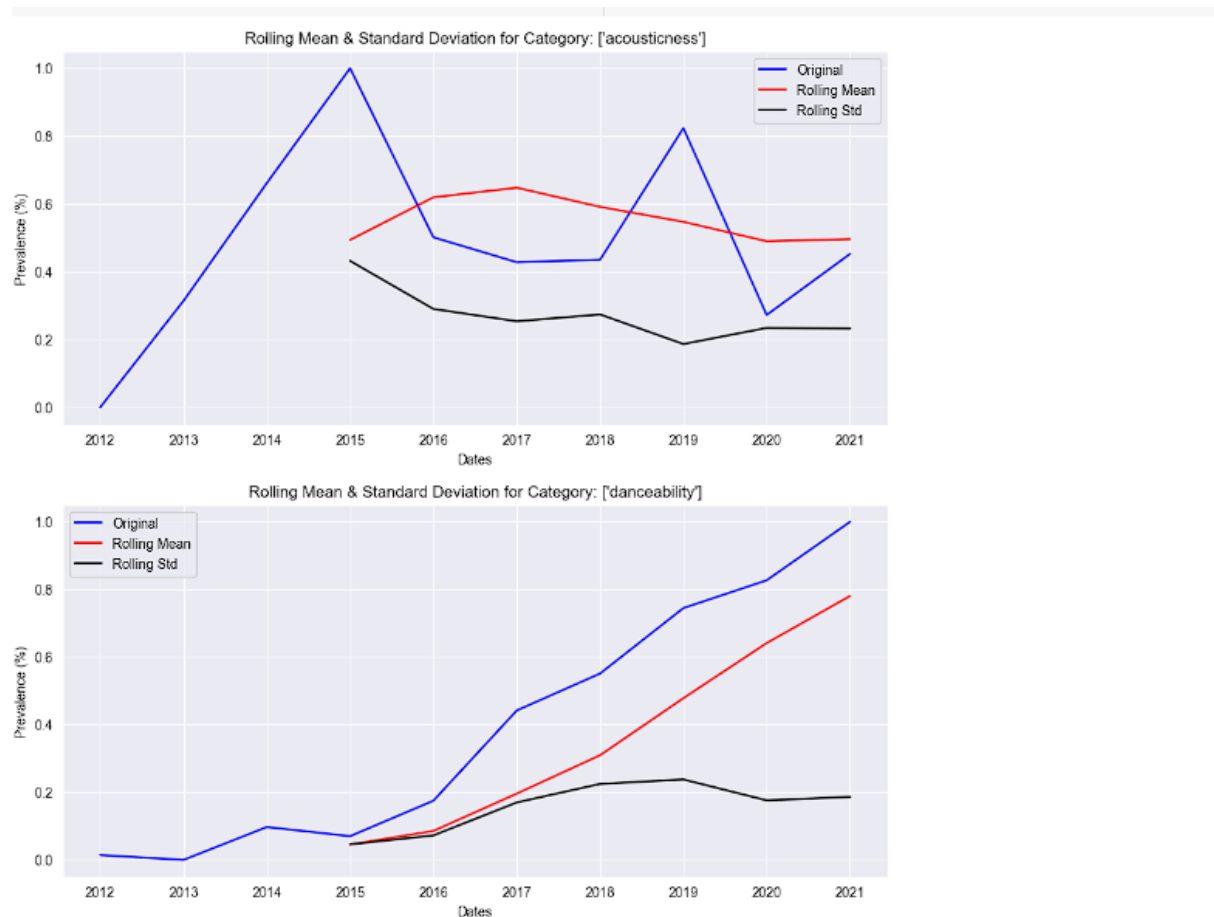
```





And you can see more graphs of different columns in the code presented by us

```
for i in range(len(df_ts)):
    rollingmean = df_ts[i].rolling(window = 4, center = False).mean()
    rollingstd = df_ts[i].rolling(window = 4, center = False).std()
    fig = plt.figure(figsize=(11,5))
    orig = plt.plot(df_ts[i], color='blue',label='Original')
    mean = plt.plot(rollingmean, color='red', label='Rolling Mean')
    std = plt.plot(rollingstd, color='black', label = 'Rolling Std')
    plt.xlabel('Dates')
    plt.ylabel('Prevalence (%)')
    plt.legend(loc='best')
    plt.title(f'Rolling Mean & Standard Deviation for Category: {df_ts[i].columns.to_list()}')
    plt.show()
```



```

acous_ts = df_ts[0] # Acousticness Prevalence Time Series
dance_ts = df_ts[1] # Loudness Prevalence Time Series
energy_ts = df_ts[2] # Energy Prevalence Time Series
inst_ts = df_ts[3]# # Valence Prevalence Time Series
live_ts = df_ts[4]# # Valence Prevalence Time Series
valence_ts = df_ts[5]# # Valence Prevalence Time Series
speech_ts = df_ts[6]# # Valence Prevalence Time Series
loudness = df_ts[7]# # Valence Prevalence Time Series

```

We can apply Arima and Sarima Here to model to see how our features fare over time

g. Conclusion

In conclusion, the use of linear regression models in analyzing Spotify data can provide valuable insights into what makes a song popular and how its features change over time. Our analysis using linear regression has allowed us to identify which song features are most strongly associated with a song's popularity and to make predictions about the popularity of new songs based on their features.

The linear regression model provides a simple and intuitive way to model the relationship between a song's features and its streaming count. By identifying which features are most strongly associated with a song's popularity, we can gain a better understanding of the factors that contribute to a song's success.

Overall, our analysis using linear regression demonstrates the value of time series analysis in understanding trends and patterns in music streaming data. The use of linear regression models can help music producers, marketers, and artists to make more informed decisions about music production, promotion, and marketing, and ultimately, to create more successful and popular music.

